The Ferrite Language

January 6, 2026

Contents

0.1 The Ferrite Language: Formal Specification

0.1.1 Overview

Ferrite is a statically typed, expression-based language with explicit ownership and borrowing semantics inspired by Rust. Its design philosophy centers on *compile-time memory safety* without requiring garbage collection or manual memory management. The semantics are defined by a combination of typing rules and small-step operational semantics, where the type system enforces memory safety through precise ownership tracking.

The language features an affine type system, meaning values can be used at most once unless they implement the Copy trait. This restriction,

combined with explicit borrowing, prevents common memory safety violations such as use-after-free, double-free, and data races at compile time.

0.1.2 Judgement Forms

The formal semantics of Ferrite are expressed through two primary judgement forms:

Typing Judgements

$$\Gamma; \Sigma \vdash e : \tau$$

This judgement reads: "Under type environment Γ and ownership environment Σ, expression e has type τ." The separation of type and ownership information is crucial—Γ tracks *what* types variables have, while Σ tracks *how* they can be used (owned, moved, or borrowed).

Evaluation Judgements

$$\langle e, \Sigma \rangle \to \langle e', \Sigma' \rangle$$

This judgement specifies that expression e in ownership state Σ evaluates in one step to expression e' in ownership state Σ'. Notably, the ownership state can change during evaluation (e.g., when a value is moved or borrowed), while types remain invariant (Preservation theorem).

0.1.3 Types

$$\tau ::= i32 \mid bool \mid unit \mid S \mid \&\tau \mid \&_{mut}\tau$$

Where S ranges over structure types (user-defined structs).

Type Descriptions

- $i32$: 32-bit signed integer (primitive, `Copy`)
- $bool$: Boolean value (primitive, `Copy`)
- $unit$: Empty type, analogous to `void` in C or `()` in Rust

- S: Struct type with named fields
- $\&\tau$: Immutable reference to a value of type τ (shared borrow)
- $\&_{mut}\tau$: Mutable reference to a value of type τ (exclusive borrow)

Reference types are *non-owning* pointers with lifetime constraints enforced at compile time. Unlike owned types, references cannot outlive their referents.

0.1.4 Environments

Ferrite maintains two distinct but coordinated environments during type checking:

Type Environment

$$\Gamma ::= \emptyset \mid \Gamma, x : \tau$$

Maps variables to their types. This is the traditional typing context found in most typed languages. Once bound, a variable's type never changes (though it may become inaccessible after being moved).

Ownership Environment

$$\Sigma ::= \emptyset \mid \Sigma, x : \omega$$

$$\omega ::= owned \mid moved \mid borrowed(n)$$

Tracks the ownership state of each variable:

- $owned$: The variable owns its value and can be freely used or moved
- $moved$: The value has been transferred elsewhere; the variable is inaccessible
- $borrowed(n)$: The value has n active immutable borrows; it can be read but not moved or mutably borrowed

The ownership environment Σ changes during evaluation and at scope boundaries, reflecting the runtime behavior of the ownership system.

0.1.5 Values

$$v ::= n \mid true \mid false \mid S(v_1, \ldots, v_n) \mid \&x \mid \&_{mut} x$$

Values are fully evaluated expressions:

- n: Integer literals
- $true$, $false$: Boolean literals
- $S(v_1, \ldots, v_n)$: Struct value with fields initialized to values
- $\&x$: Immutable reference to variable x
- $\&_{mut} x$: Mutable reference to variable x

Note that references are *syntactic*: they refer to variables in scope, not arbitrary heap addresses. This simplification enables straightforward lifetime checking.

0.1.6 Typing Rules

The typing rules define when an expression is well-typed. Each rule has premises (above the line) that must hold for the conclusion (below the line) to be valid.

Variables

$$\frac{\Gamma(x) = \tau \quad \Sigma(x) = owned}{\Gamma; \Sigma \vdash x : \tau} \quad \text{T-Var}$$

A variable can be used only if it is currently owned. This rule prevents use-after-move errors. Note that using a variable in an owned context will trigger a move during evaluation (see operational semantics).

Copy Types

$$\frac{\Gamma(x) = \tau \quad \tau \in Copy \quad \Sigma(x) \neq moved}{\Gamma; \Sigma \vdash x : \tau} \quad \text{T-Var-Copy}$$

For types implementing Copy (primitives and references), the variable can be used multiple times without moving. The ownership state remains unchanged.

Let Binding

$$\frac{\Gamma; \Sigma \vdash e_1 : \tau_1 \quad \Gamma[x : \tau_1]; \Sigma[x \mapsto \mathit{owned}] \vdash e_2 : \tau_2}{\Gamma; \Sigma \vdash (\texttt{let}\ ((x\ e_1))\ e_2) : \tau_2} \quad \text{T-Let}$$

A let-binding evaluates e_1 to produce a value, binds it to x with ownership, then evaluates the body e_2 in the extended environment. The type of the entire expression is the type of the body. If e_1 consumes any variables, they are marked as moved in Σ before type-checking e_2.

Struct Construction

$$\frac{\Gamma; \Sigma \vdash e_i : \tau_i \quad \forall i \quad \mathit{fields}(S) = \{f_i : \tau_i\}}{\Gamma; \Sigma \vdash (S\ e_1\ \ldots\ e_n) : S} \quad \text{T-Struct}$$

Constructing a struct requires providing expressions for each field. Each argument e_i is evaluated and its value is *moved* into the struct (unless it has Copy type). After construction, any non-Copy variables used in e_i are marked as moved.

Field Access

$$\frac{\Gamma; \Sigma \vdash e : S \quad \mathit{fields}(S)(f) = \tau}{\Gamma; \Sigma \vdash (.\ e\ f) : \tau} \quad \text{T-Field}$$

Accessing a field of a struct returns a value of the field's type. If e is an owned struct, the entire struct is consumed (moved). If e is a reference $\&S$ or $\&_{mut}S$, the field access borrows from the reference.

Immutable Borrowing

$$\frac{\Gamma(x) = \tau \quad \Sigma(x) = \mathit{owned}}{\Gamma; \Sigma[x \mapsto \mathit{borrowed}(1)] \vdash (\texttt{borrow}\ x) : \&\tau} \quad \text{T-Borrow}$$

$$\frac{\Gamma(x) = \tau \quad \Sigma(x) = \mathit{borrowed}(n)}{\Gamma; \Sigma[x \mapsto \mathit{borrowed}(n+1)] \vdash (\texttt{borrow}\ x) : \&\tau} \quad \text{T-Borrow-Shared}$$

Borrowing creates an immutable reference without transferring ownership. Multiple immutable borrows can coexist (incrementing the borrow count). While borrowed, the original variable cannot be moved or mutably borrowed, preventing use-after-free and data races.

Mutable Borrowing

$$\frac{\Gamma(x) = \tau \quad \Sigma(x) = owned}{\Gamma; \Sigma[x \mapsto borrowed_{mut}] \vdash (\texttt{borrow-mut}\ x) : \&_{mut}\tau} \quad \text{T-Borrow-Mut}$$

A mutable borrow is *exclusive*: no other borrows (mutable or immutable) can exist simultaneously. This guarantees that mutations through the reference cannot create data races or iterator invalidation.

Function Application (Move Semantics)

$$\frac{\Gamma; \Sigma \vdash f : (\tau_1 \ldots \tau_n \to \tau_r) \quad \Gamma; \Sigma_0 \vdash e_i : \tau_i \quad \Sigma_{i+1} = \Sigma_i[e_i \mapsto moved]}{\Gamma; \Sigma_n \vdash (f\ e_1 \ldots e_n) : \tau_r} \quad \text{T-Call}$$

When calling a function with owned parameters, arguments are moved into the function. Each argument evaluation potentially changes the ownership state (sequentially threaded through Σ_i). After the call, moved variables are inaccessible.

Function Application (Borrow Semantics)

$$\frac{\Gamma; \Sigma \vdash f : (\&\tau \to \tau_r) \quad \Gamma; \Sigma \vdash e : \&\tau}{\Gamma; \Sigma \vdash (f\ e) : \tau_r} \quad \text{T-Call-Borrow}$$

Functions accepting references do not consume their arguments. The caller retains ownership, and the ownership state Σ is unchanged (aside from any borrow state changes from creating the reference).

0.1.7 Operational Semantics

Ferrite uses call-by-value evaluation with explicit ownership transitions. The operational semantics model how expressions reduce to values and how ownership states evolve during execution.

Variable Evaluation (Move)

$$\frac{\Sigma(x) = owned}{\langle x, \Sigma \rangle \to \langle v, \Sigma[x \mapsto moved] \rangle} \quad \text{E-Var-Move}$$

Reading an owned variable *moves* its value, leaving the variable in a moved state. This implements affine types: each owned value is used at most once.

Variable Evaluation (Copy)

$$\frac{\Sigma(x) = owned \quad \Gamma(x) = \tau \quad \tau \in Copy}{\langle x, \Sigma\rangle \to \langle v, \Sigma\rangle} \quad \text{E-Var-Copy}$$

For Copy types, reading a variable produces a copy of the value without affecting the ownership state.

Borrow Evaluation

$$\langle(\texttt{borrow}\ x), \Sigma\rangle \to \langle \&x, \Sigma[x \mapsto borrowed(n+1)]\rangle \quad \text{E-Borrow}$$

Creating a borrow produces a reference value and increments the borrow count. The variable remains accessible for further immutable borrows.

Let Evaluation (Step)

$$\frac{\langle e_1, \Sigma\rangle \to \langle e_1', \Sigma'\rangle}{\langle(\texttt{let}\ ((x\ e_1))\ e_2), \Sigma\rangle \to \langle(\texttt{let}\ ((x\ e_1'))\ e_2), \Sigma'\rangle} \quad \text{E-Let-Step}$$

If the bound expression can take a step, the let-expression steps by evaluating the binding expression.

Let Evaluation (Bind)

$$\langle(\texttt{let}\ ((x\ v))\ e), \Sigma\rangle \to \langle e[x \mapsto v], \Sigma[x \mapsto owned]\rangle \quad \text{E-Let-Bind}$$

Once the bound expression is a value, substitute it into the body and mark the variable as owned in the ownership environment.

Struct Construction

$$\frac{\forall i : \langle e_i, \Sigma_{i-1}\rangle \to \langle v_i, \Sigma_i\rangle}{\langle(S\ e_1 \ldots e_n), \Sigma_0\rangle \to \langle S(v_1, \ldots, v_n), \Sigma_n\rangle} \quad \text{E-Struct}$$

Struct construction evaluates each field expression left-to-right, threading ownership state changes through the sequence.

0.1.8 Scope Exit

When execution exits a scope (e.g., at the end of a let-body), any active borrows are released:

$$\Sigma(x) = borrowed(n) \Rightarrow \Sigma'(x) = \begin{cases} owned & \text{if } n = 1 \\ borrowed(n-1) & \text{if } n > 1 \end{cases}$$

This models the automatic release of borrows when their lexical scope ends. If the variable was moved or is still borrowed elsewhere, it retains that state.

0.1.9 Type Soundness

Type soundness is the fundamental safety property: well-typed programs do not get stuck in undefined states. We prove this through the standard progress and preservation approach.

Lemma 1 (Canonical Forms)

If $\Gamma; \Sigma \vdash v : \tau$ and v is a value, then:

- If $\tau = i32$, then $v = n$ for some integer n
- If $\tau = bool$, then $v \in \{true, false\}$
- If $\tau = S$, then $v = S(v_1, \ldots, v_n)$ where each v_i is a value
- If $\tau = \&\tau'$, then $v = \&x$ for some variable x with $\Gamma(x) = \tau'$

Proof Sketch By case analysis on the typing derivation. Each typing rule for values ensures the value has the expected form. For example, T-Borrow is the only rule that produces type $\&\tau$, and it requires the expression to be $(\texttt{borrow}\ x)$, which evaluates to $\&x$.

Theorem 1 (Progress)

If $\emptyset; \Sigma \vdash e : \tau$, then either:

1. e is a value, or
2. there exist e', Σ' such that $\langle e, \Sigma \rangle \to \langle e', \Sigma' \rangle$

Proof Sketch By induction on the typing derivation of e.

Base cases:

- Literals (n, $true$, $false$) are values
- Variables: By T-Var, $\Sigma(x) = owned$, so E-Var-Move applies

Inductive cases:

- Let-binding: By induction, e_1 is either a value (so E-Let-Bind applies) or steps (so E-Let-Step applies)
- Borrow: By typing rule, variable is owned or borrowed, so E-Borrow applies
- Function call: By induction, arguments step or are values; when all are values, the function body steps

Well-typed expressions cannot get stuck because ownership guarantees in Σ align exactly with the preconditions of evaluation rules.

Theorem 2 (Preservation)

If $\Gamma; \Sigma \vdash e : \tau$ and $\langle e, \Sigma \rangle \to \langle e', \Sigma' \rangle$, then $\Gamma; \Sigma' \vdash e' : \tau$.

Proof Sketch By induction on the evaluation step.

Key cases:

- E-Var-Move: Type unchanged (τ), ownership updated to $moved$. Since the variable is no longer accessible, this preserves well-typedness.
- E-Borrow: Type becomes $\&\tau$, ownership becomes $borrowed(n+1)$. The typing rule T-Borrow explicitly allows this transition.

- E-Let-Bind: Substitution lemma ensures $e[x \mapsto v]$ has the same type as e under the extended environment. The ownership state correctly reflects that x is now owned.

- E-Struct: Each field steps preserving its type, so the final struct value has the correct type.

The critical insight is that ownership transitions in Σ mirror the constraints in the typing rules, so type judgements are preserved across evaluation steps.

Corollary (Type Soundness)

Well-typed Ferrite programs do not exhibit:

- **Use-after-move**: Variables marked as $moved$ cannot be used (enforced by T-Var)

- **Dangling references**: References cannot outlive their referents (enforced by lexical scoping and borrow rules)

- **Data races**: Mutable borrows are exclusive; immutable borrows prevent mutation

- **Invalid field access**: Field access requires the struct type to define that field

Proof These properties follow directly from Progress (no well-typed program gets stuck) and Preservation (types and ownership are maintained during execution). Any violation would require either a stuck state (contradicting Progress) or a type mismatch after a step (contradicting Preservation).

0.1.10 Traits and Polymorphism

Ferrite supports trait-based polymorphism, enabling generic programming while maintaining type safety.

Trait Definitions A trait T declares a set of method signatures:

$$trait\ T = \{m_i : \tau_i\}$$

Trait Implementation A type S can implement trait T by providing definitions for all methods:

$$impl\ T\ for\ S$$

Trait Bounds Function types can be constrained by trait bounds:

$$\forall \alpha : T.(\alpha \to \tau)$$

This reads: "for any type α implementing trait T, a function from α to τ."

Built-in Traits

- $Copy$: Types whose values can be duplicated bitwise
- $Clone$: Types that support explicit cloning
- $Debug$: Types that can be formatted for debugging

0.1.11 Conclusion

Ferrite's safety guarantees arise from the synergy of three key design elements:

1. **Affine types with Copy**: Values are used at most once unless explicitly marked as copyable, preventing use-after-free at compile time.
2. **Explicit borrow tracking**: References are first-class, with ownership states enforcing aliasing XOR mutability (you can have many immutable borrows OR one mutable borrow, but not both).
3. **Operational ownership transitions**: The operational semantics directly model ownership transfer and borrow creation, making the runtime behavior of the ownership system explicit and verifiable.

The language is both **implementable** (the type system is decidable and compiles to efficient C code) and **formally sound** (proven memory-safe via Progress and Preservation). This makes Ferrite suitable for systems programming where safety and performance are both critical requirements.

Future extensions could include:

- Lifetime parameters for more flexible borrowing patterns
- Higher-kinded types for advanced trait abstractions
- Effect systems for tracking computational effects (I/O, exceptions)
- Dependent types for richer compile-time guarantees